Binaml: Continual Learning over Binary Feature Streams

Romain Zimmer
hi@romainzimmer.com

2026-08-11 (Draft)

Abstract

Binaml focuses on continual learning models over streams of binary features subject to data drift. Binary features provide a task-agnostic representation and enable models optimized for memory, latency, and energy efficiency. This paper specifies Binaml's current regression model, `BRegressor`, as an initial task-specific instantiation. It adapts online rather than relying on a stationary train/serve assumption and learns compact compositional representations from residual feedback. The model is environment-agnostic: it does not depend on how the stream is produced. This paper specifies the current regression model and reports reproducible prequential comparisons on versioned synthetic streams.

1 Notation

d, x_t, y_t Input width, binary input, and scalar regression target.

$\hat{y}_t, e_t, s_t$ Prediction, residual, and residual-sign label.

b, w_r, η, λ Intercept, coefficient of feature id r, learning rate, and ℓ_2 decay.

$r = (\ell, j)$ Feature id with store layer ℓ and index j.

$\varphi_r, \psi_r, \mathcal{L}$ Boolean value, centered value, and live feature set for feature id r.

$T \in \{0, \ldots, 15\}$ Four-bit truth table of an arity-2 composed feature.

r_0, r_1 Ordered input feature ids of a composed feature.

B, S, n, K Replay-batch size, SGD steps per observation, feature-batch index, and parent top-K width.

C_ℓ, V_ℓ, L Live capacity per learned layer, candidate capacity per layer, and maximum learned-layer depth.

$\sigma(r), \tilde{\sigma}(r)$ Match score and propagated pruning score.

2 Model

At time t, the model receives $x_t \in \{0,1\}^d$ and later observes a finite scalar y_t. If useful signal is sparse weighted boolean structure over bits, a linear head over learned boolean indicators is a direct hypothesis class: continuous amplitudes, discrete features, and an inspectable composition. Let $\mathcal{L}$ be the current set of live feature ids. The prediction is

$$\hat{y}_t = b + \sum_{r \in \mathcal{L}} w_r \, \psi_r(x_t),$$

where $b \in \mathbb{R}$, each $w_r \in \mathbb{R}$, and each $\varphi_r(x_t) \in \{0, 1\}$ is centered before entering the linear head:

$$\psi_r(x_t) = 2\varphi_r(x_t) - 1 \in \{-1, 1\}.$$

Layer zero. Source coordinates are immutable feature ids $r = (0, j)$ for $j \in \{0, \ldots, d-1\}$, with

$$\varphi_{(0,j)}(x) = x_j.$$

Their initial coefficients are zero.

Composed features. Composition starts from the most basic nontrivial boolean learning unit: two-input tables. Combining them yields richer features without beginning from high-arity search. A composed feature combines two input features (r_0, r_1) using a four-bit truth table $T \in \{0, \ldots, 15\}$.

3 Feature representation and storage

Depth is the natural complexity axis for feature composition: it bounds how far candidates look, keeps evaluation a DAG walk, and makes capacity control per depth band. The store layer of a composed feature is

$$\ell(r) = 1 + \max(\ell(r_0), \ell(r_1)).$$

Layer zero holds the d source features. Each learned layer $\ell \geq 1$ maintains two collections:

Live Optionally occupied slots of composed features. Indices are stable; pruned slots become tombstones and are no longer live feature ids.

Candidates Candidate features trained on the current batch and not yet admitted to the live set.

A candidate insert requires ordered distinct inputs $r_0 < r_1$, valid live or source parents, and the correct derived layer. The store permits multiple features with the same parent pair, allowing truth tables learned on different batches to coexist. Newly promoted live coefficients start at zero. After promote or prune, coefficients of dead feature ids are dropped and missing live feature ids receive weight zero.

3.1 Runtime layout

The implementation separates stable feature identity from compact evaluation. Each id is the pair `FeatureId(layer, index)`. Source scores occupy one contiguous vector. Learned layers occupy a vector of layer records; each record has a live vector of optional feature records and a candidate vector. An absent live entry is a tombstone, so pruning does not renumber an id that a later feature may reference. A feature record stores two parent ids, one four-bit `u8` truth table, and one `u8` match score. The eight `u8` counters used to learn a table cover every $(u, v, s) \in \{0, 1\}^3$ combination, which limits the feature-learning batch size to 255.

Before evaluation, the live store is flattened in layer order into a replay layout: one vector of feature ids and one same-length vector of nodes. A source node stores its input coordinate; a composed node stores the two already evaluated node positions and its truth table. Thus each input is evaluated in one forward topological pass. The replay cache is a bounded deque of these `u8` activation rows, alongside deques of the original binary inputs and targets. Coefficients are kept separately in a map from feature id to `f64`; after each structural update, tombstoned weights are removed and new live ids receive weight zero.

Type	**Fields or representation**	**Role**
FeatureId	{layer: usize, index: usize}	stable source or learned-feature identity
Feature	{inputs: [FeatureId; 2], truth_table: u8, score: u8}	composed boolean feature and pruning score
FeatureLayer	{live: Vec<Option<Feature», candidates: Vec<Feature>}	one learned depth band; empty live slots are tombstones
FeatureStore	{source_scores: Vec<u8>, layers: Vec<FeatureLayer>}	owns all source, live, and candidate features
FeatureLearningConfig	{batch_size, parent_top_k, features_per_layer, candidate_capacity, max_layers}	feature-search limits and parent-selection width
FeatureLearner	{config, store, has_previous_batch: bool}	scores, promotes, prunes, and generates features
ReplayLayout	{references: Vec<FeatureId>, nodes: Vec<FeatureNode>}	compact layer-ordered evaluation plan
FeatureNode	Source(usize) or Composed{first: usize, second: usize, truth_table: u8}	source coordinate or compiled composed operation
ReplayCache	{layout: ReplayLayout, rows: VecDeque<Vec<u8»}	bounded cached activation rows
BRegressor	learner, coefficient HashMap<FeatureId, f64>, intercept f64, replay input/target deques, cache, count	online model state

4 Online linear-head updates

Composed-feature identity is discrete; coefficients are continuous, so the two are updated separately. The model retains the most recent B observations. For each new observation, it performs S full-batch squared-error gradient steps over that replay window. For one such step, let $e_i = y_i - \hat{y}_i$ be each pre-step residual and $\mathcal{R}$ be the current replay window. With learning rate $\eta > 0$ and decay $\lambda \geq 0$, apply the averaged data gradient with multiplicative ℓ_2 shrinkage:

$$\begin{aligned} w_r &\leftarrow (1-\eta\lambda)\, w_r && \text{for all live } r, \\ b &\leftarrow b + \eta \frac{1}{|\mathcal{R}|} \sum_{i\in\mathcal{R}} e_i, \\ w_r &\leftarrow w_r + \eta \frac{1}{|\mathcal{R}|} \sum_{i\in\mathcal{R}} e_i \psi_r(x_i). \end{aligned}$$

There is no gradient through boolean structure: SGD tracks residual amplitude at the batch level, while structure search runs when enough residual-sign evidence exists. The residual signs

$$s_i = \mathbf{1}\{e_i \geq 0\}$$

are computed before the associated replay updates. After every B new observations, the ring buffer has been completely refreshed; the learner consumes that non-overlapping window for feature learning.

5 Residual-sign supervision and batching

Feature learning targets residual signs, not y_t itself. Residual signs are a binary supervision bit for the discrete learner, while the linear head absorbs magnitude—simple parts combined, not one joint high-complexity update. Exact truth-table counting needs a bounded window; batching amortizes candidate search and gives a natural train/hold-out split across successive windows. Batch n is

$$\mathcal{B}_n = \{(x_t, s_t)\}_{t\in I_n}, \qquad |I_n| = B,$$

where I_n is the contiguous observation window that completely refreshed the ring buffer. Processing $\mathcal{B}_n$ has three phases when a previous batch exists:

1. Score every source, live feature, and candidate on $\mathcal{B}_n$ by match count against residual signs.
2. Promote every candidate into live, then prune each learned layer to capacity C_ℓ.
3. Learn new candidates on $\mathcal{B}_n$.

On the first batch only the candidate-learning phase runs. Candidates trained on $\mathcal{B}_n$ therefore receive their first scores on $\mathcal{B}_{n+1}$ before promotion: a one-batch hold-out across successive residual-sign windows.

Match score. For a boolean stream $z_{1:B}$ and residual signs $s_{1:B}$,

$$\sigma = \sum_{i=1}^{B} \mathbf{1}\{z_i = s_i\}.$$

Sources use their coordinate columns; composed features use their evaluated bitstreams.

6 Truth-table learning

Given two parent bitstreams $u_{1:B}$, $v_{1:B}$ and residual signs $s_{1:B}$, the learner returns the Hamming-error-minimizing four-bit table. Its state is a fixed array of eight `u8` counters, one for each combination of the two parent values and residual sign. The learned result is a four-bit truth table represented by one `u8`. The counters record the frequencies needed to select each table output by majority, with ties resolving to zero. As one counter may receive every observation, the `u8` counters bound the admissible batch size to $B \leq 255$.

7 Candidate feature generation

For each layer $\ell = 1, \ldots, L$, while candidate capacity V_ℓ remains:

1. Let P be the top-K live feature ids in layer $\ell - 1$ by match score, breaking ties by feature-id order.
2. Form every distinct pair $\{r_a, r_b\} \subset P$, written with $r_0 < r_1$.
3. For each pair, learn a truth table on the current batch and insert it into candidates with initial score zero, stopping if candidate capacity is reached. A pair may produce a new candidate even when that pair already appears in the layer.

If P has fewer than two feature ids, layer ℓ is skipped. Layer depth never exceeds L.

8 Feature promotion and pruning

After scoring on the following batch, every candidate at each learned layer is appended to that layer's live list. A parent can look weak alone yet be required by useful children, so pruning uses max-with-descendants scores. If a layer then exceeds capacity C_ℓ, live feature ids are sorted by ascending propagated score

$$\tilde{\sigma}(r) = \max\Big(\sigma(r), \max_{r' \in \mathrm{children}(r)} \tilde{\sigma}(r')\Big),$$

where $\mathrm{children}(r)$ are live features that take r as an input, and ties break by feature-id order. The lowest $|\mathrm{live}| - C_\ell$ feature ids are tombstoned. Tombstoning cascades to all dependents, so no live feature retains a dead parent.

9 Online learning procedure

Configuration

Linear Learning rate η, decay λ, and S replay SGD steps per observation.

Batch structure Batch size B, parent top-K width K, per-layer live capacity C_ℓ, candidate capacity V_ℓ, and maximum depth L.

Initialize: empty learned layers, $b \leftarrow 0$, $w_{(0,j)} \leftarrow 0$ for all source indices, and an empty replay ring buffer.
For each observation (x_t, y_t):

1. Validate the input width and finite target. If the replay cache is full, evict its oldest input, target, and activation row.
2. Evaluate live $\varphi_r(x_t)$ through the current replay layout, predict $\hat{y}_t = b + \sum_{r \in \mathcal{L}} w_r \varphi_r(x_t)$, and append the input, target, and activation row.
3. At every Bth observation, compute residual signs from the current replay rows before any coefficient update.
4. Apply S full-batch linear updates over the current ring.
5. After a complete refresh, score the previous candidates, promote and prune live features, then generate candidates from the current residual-sign batch. Synchronize coefficients and rebuild the replay layout and its cached activation rows.

Cost. Prediction and cache insertion are linear in the number of live source and composed features. Each linear step costs the replay-window size times that same live-feature count. Structural work occurs once per complete refresh: candidate table learning scans each selected parent pair over B rows, while pruning recursively scans later layers to preserve a useful descendant's parents. The design trades occasional tombstones for stable ids and bounded live and candidate capacity per layer.

10 API and integration boundaries

The `binaml.models` package owns the online model and has no environment dependency: the goal is a generic model, decoupled from where the data comes from and how it was produced. The public surface is `BRegressor`, which implements `predict(features)` and `observe(features, target)` for binary feature vectors of width d. The prediction call may retain pairing state; `observe` must follow a preceding `predict`. Feature learning is internal to `observe` once a batch fills. Evaluation packages compose models with environments through predict-then-observe prequential evaluation. Named benchmarks only compose these independent components.

11 Prequential evaluation protocol

For each emitted (x_t, y_t), an evaluator first records the model prediction, then exposes the target through `observe`. Mean squared error is computed from the recorded pre-update predictions. Structure updates occur only after a full replay-ring refresh inside `observe`; their residual signs are measured before the corresponding batch-gradient updates. Hyperparameters, implementation version, and the online protocol are required to interpret any reported trajectory.

12 Evaluation

The benchmark inputs and model configuration are versioned in this paper's benchmark directory. The task uses 1,000 observations and seeds $\{0, 1, 2, 3, 4\}$ with 32 input bits, 12 fixed target components, noise standard deviation 0.1, and the same function, DAG, and target-scale settings. Input, gate, and intercept distributions each resample independently with probability 0.01.

All models receive the scenario width. The feature regressor uses learning rate 0.03, ℓ_2 decay 10^{-4}, replay size 32, three full-batch steps, parent top-$K = 8$, 32 live features per learned layer, candidate capacity 32, and depth two. Every source and composed activation enters the linear head as $2\varphi_r(x) - 1$, and candidate pairs are distinct members of the top-K previous layer only. The linear baseline uses the same learning rate, decay, replay size, and step count, with uncentered binary inputs. The MLP baseline uses replay size 32 and three `partial_fit` passes; one 50-unit ReLU hidden layer; scikit-learn SGD with constant learning rate 0.003, $\alpha = 10^{-4}$, `power_t`=0.5, shuffling enabled, random state zero, tolerance 10^{-4}, zero momentum, no Nesterov momentum, and no early stopping. Its remaining configured values are validation fraction 0.1, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, 10 no-change iterations, and disabled verbose output; the wrapper fixes one iteration and disables warm start.

Tables report the mean ± standard error over five seeds. MSE is the optimized loss; RMSE expresses the same error in target units. Total time is the prequential predict-plus-observe time for a 1,000-sample trajectory.

Model	MSE	RMSE	total s
BRegressor (ours)	$\mathbf{0.681 \pm 0.015}$	$\mathbf{0.825 \pm 0.009}$	0.454 ± 0.007
Linear SGD	0.744 ± 0.020	0.862 ± 0.011	$\mathbf{0.153 \pm 0.002}$
MLP SGD	0.845 ± 0.031	0.919 ± 0.017	3.684 ± 0.389

13 Limitations and scope

This document specifies the feature regressor used by Binaml. It does not claim optimality of residual-sign learning. The reported comparison is limited to the versioned synthetic task, fixed configurations, and reproducible baseline runs described above.